DYNAMOAI

COMPASS AI

From AI Ideas to Production Systems.

AI Engineering • Backend • Agents • RAG • MCP • System Design • Production

06 · Flagship Course — End-to-End AI Engineering

Subtitle: From Python and Backend Engineering to Production AI Agents

Each module below is a sub-tab. Update Status / Script / Code / Published as you progress.

Module 0 — AI Engineering Foundations

Lesson	Status	Script	Code	Published
Welcome to the Course	💡 Idea		—	—
What Is an AI Engineer?	💡 Idea	—	—	—
Anatomy of a Modern AI Application	💡 Idea	—	—	—
AI Engineering Stack	💡 Idea	—	—	—
Course Roadmap	💡 Idea	—	—	—

Module 1 — Python for AI Engineers

Lesson	Status	Script	Code	Published
Python prerequisites	💡 Idea	—	—	—
Python project structure	💡 Idea	—	—	—
Functions	💡 Idea	—	—	—
Type hints	💡 Idea	—	—	—
Classes & OOP	💡 Idea	—	—	—
Dataclasses	💡 Idea	—	—	—
Pydantic	💡 Idea	—	—	—
Error handling	💡 Idea	—	—	—
JSON	💡 Idea	—	—	—
HTTP requests	💡 Idea	—	—	—
Environment variables	💡 Idea	—	—	—
Virtual environments	💡 Idea	—	—	—
Dependencies	💡 Idea	—	—	—
Async Python	💡 Idea	—	—	—
async / await	💡 Idea	—	—	—
Testing with pytest	💡 Idea	—	—	—
Git & GitHub essentials	💡 Idea	—	—	—

Module Project: Python API client communicating with an AI API.

MATERIALS: LINK

Module 2 — Backend Engineering

Lesson	Status	Script	Code	Published
How the Internet Works	💡 Idea	—	—	—
HTTP	💡 Idea	—	—	—
REST APIs	💡 Idea	—	—	—
API Design	💡 Idea	—	—	—
FastAPI	💡 Idea	—	—	—
Routes	💡 Idea	—	—	—
Request Validation	💡 Idea	—	—	—
Response Models	💡 Idea	—	—	—
Dependency Injection	💡 Idea	—	—	—
Authentication	💡 Idea	—	—	—
Authorization	💡 Idea	—	—	—
PostgreSQL	💡 Idea	—	—	—
SQL	💡 Idea	—	—	—
Database Design	💡 Idea	—	—	—
SQLAlchemy	💡 Idea	—	—	—
Database Migrations	💡 Idea	—	—	—
Redis	💡 Idea	—	—	—
Caching	💡 Idea	—	—	—
Background Jobs	💡 Idea	—	—	—
Queues	💡 Idea	—	—	—
Async Backend Architecture	💡 Idea	—	—	—

Module Project: Production-style FastAPI backend with PostgreSQL and Redis.

MATERIALS: LINK

Module 3 — LLM Fundamentals

Lesson	Status	Script	Code	Published
What Are LLMs?	💡 Idea	—	—	—
How LLMs Work	💡 Idea	—	—	—
Tokens	💡 Idea	—	—	—
Context Windows	💡 Idea	—	—	—
Temperature	💡 Idea	—	—	—
Sampling	💡 Idea	—	—	—
System Prompts	💡 Idea	—	—	—
User Prompts	💡 Idea	—	—	—
Prompt Engineering	💡 Idea	—	—	—
Structured Outputs	💡 Idea	—	—	—
LLM APIs	💡 Idea	—	—	—
Streaming	💡 Idea	—	—	—
LLM Errors	💡 Idea	—	—	—
Model Selection	💡 Idea	—	—	—
Open vs Closed Models	💡 Idea	—	—	—

Module Project: Build an AI-powered API.

MATERIALS: LINK

Module 4 — LLM Application Engineering

Lesson	Status	Script	Code	Published
LLM Application Architecture	💡 Idea	—	—	—
Prompt Templates	💡 Idea	—	—	—
Structured Data Extraction	💡 Idea	—	—	—
Function Calling	💡 Idea	—	—	—
Tool Calling	💡 Idea	—	—	—
Model Routing	💡 Idea	—	—	—
Multiple Providers	💡 Idea	—	—	—
Fallback Strategies	💡 Idea	—	—	—
Streaming Architecture	💡 Idea	—	—	—
Token Management	💡 Idea	—	—	—
Cost Management	💡 Idea	—	—	—
Context Management	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Error Handling	💡 Idea	—	—	—
Production LLM Service	💡 Idea	—	—	—

Module Project: AI-powered document-processing API.

Module 5 — RAG

Lesson	Status	Script	Code	Published
Why LLMs Hallucinate	💡 Idea	—	—	—
What Is RAG?	💡 Idea	—	—	—
RAG Architecture	💡 Idea	—	—	—
Embeddings	💡 Idea	—	—	—
Vector Databases	💡 Idea	—	—	—
Document Ingestion	💡 Idea	—	—	—
Document Processing	💡 Idea	—	—	—
Chunking	💡 Idea	—	—	—
Embedding Generation	💡 Idea	—	—	—
Retrieval	💡 Idea	—	—	—
Metadata Filtering	💡 Idea	—	—	—
Similarity Search	💡 Idea	—	—	—
Hybrid Search	💡 Idea	—	—	—
Reranking	💡 Idea	—	—	—
Context Construction	💡 Idea	—	—	—
RAG Prompting	💡 Idea	—	—	—
RAG Evaluation	💡 Idea	—	—	—
Common RAG Failures	💡 Idea	—	—	—
Production RAG Architecture	💡 Idea	—	—	—

Module Project: Production RAG application that lets users chat with their documents.

Module 6 — AI Agents

Lesson	Status	Script	Code	Published
What Is an AI Agent?	💡 Idea	—	—	—
Agent vs Chatbot	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Agent vs Workflow	💡 Idea	—	—	—
Anatomy of an Agent	💡 Idea	—	—	—
Agent Loops	💡 Idea	—	—	—
Tool Calling	💡 Idea	—	—	—
Designing Agent Tools	💡 Idea	—	—	—
Tool Execution	💡 Idea	—	—	—
Agent State	💡 Idea	—	—	—
Agent Memory	💡 Idea	—	—	—
Short-Term Memory	💡 Idea	—	—	—
Long-Term Memory	💡 Idea	—	—	—
Planning	💡 Idea	—	—	—
Human-in-the-Loop	💡 Idea	—	—	—
Agent Error Handling	💡 Idea	—	—	—
Agent Security	💡 Idea	—	—	—
Build an Agent From Scratch	💡 Idea	—	—	—

Module Project: AI Research Agent capable of searching, reasoning, and generating structured reports.

Module 7 — Agent Orchestration

Lesson	Status	Script	Code	Published
Why Agent Frameworks Exist	💡 Idea	—	—	—
LangChain	💡 Idea	—	—	—
LangGraph	💡 Idea	—	—	—
Workflows vs Agents	💡 Idea	—	—	—
Graph-Based Agent Architecture	💡 Idea	—	—	—
Nodes	💡 Idea	—	—	—
Edges	💡 Idea	—	—	—
Conditional Routing	💡 Idea	—	—	—
State Management	💡 Idea	—	—	—
Persistence	💡 Idea	—	—	—
Checkpointing	💡 Idea	—	—	—
Streaming	💡 Idea	—	—	—
Human Approval	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Multi-Agent Systems	💡 Idea	—	—	—
Supervisor Architecture	💡 Idea	—	—	—
Agent Collaboration	💡 Idea	—	—	—
Production Agent Graph	💡 Idea	—	—	—

Module Project: Multi-agent research system.

Module 8 — MCP

Lesson	Status	Script	Code	Published
What Is MCP?	💡 Idea	—	—	—
Why MCP Exists	💡 Idea	—	—	—
MCP Architecture	💡 Idea	—	—	—
MCP Clients	💡 Idea	—	—	—
MCP Servers	💡 Idea	—	—	—
MCP Tools	💡 Idea	—	—	—
MCP Resources	💡 Idea	—	—	—
MCP Prompts	💡 Idea	—	—	—
Build an MCP Server	💡 Idea	—	—	—
Connect an Agent to MCP	💡 Idea	—	—	—
MCP Security	💡 Idea	—	—	—
Production MCP Architecture	💡 Idea	—	—	—

Module Project: MCP-powered AI agent with external tools.

Module 9 — Production AI Engineering

Lesson	Status	Script	Code	Published
Prototype → Production	💡 Idea	—	—	—
Production AI Architecture	💡 Idea	—	—	—
Project Structure	💡 Idea	—	—	—
Configuration Management	💡 Idea	—	—	—
Secrets Management	💡 Idea	—	—	—
Logging	💡 Idea	—	—	—
Error Handling	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Retries	💡 Idea	—	—	—
Rate Limiting	💡 Idea	—	—	—
Caching	💡 Idea	—	—	—
Authentication	💡 Idea	—	—	—
Authorization	💡 Idea	—	—	—
AI Security	💡 Idea	—	—	—
Prompt Injection	💡 Idea	—	—	—
Guardrails	💡 Idea	—	—	—
Output Validation	💡 Idea	—	—	—
LLM Evaluation	💡 Idea	—	—	—
Agent Evaluation	💡 Idea	—	—	—
Testing AI Systems	💡 Idea	—	—	—
Regression Testing	💡 Idea	—	—	—
Observability	💡 Idea	—	—	—
Tracing	💡 Idea	—	—	—
Monitoring	💡 Idea	—	—	—
AI Cost Monitoring	💡 Idea	—	—	—
Prompt Versioning	💡 Idea	—	—	—
Model Versioning	💡 Idea	—	—	—

Module 10 — Docker & Deployment

Lesson	Status	Script	Code	Published
Why Docker?	💡 Idea	—	—	—
Docker Fundamentals	💡 Idea	—	—	—
Dockerfile	💡 Idea	—	—	—
Containerizing FastAPI	💡 Idea	—	—	—
PostgreSQL + Redis + API	💡 Idea	—	—	—
Docker Compose	💡 Idea	—	—	—
Production Containers	💡 Idea	—	—	—
CI/CD	💡 Idea	—	—	—
Environment Management	💡 Idea	—	—	—
Cloud Deployment	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Scaling AI Applications	💡 Idea	—	—	—
Production Architecture	💡 Idea	—	—	—

Module 11 — AI System Design

Lesson	Status	Script	Code	Published
AI System Design	💡 Idea	—	—	—
Designing for Scale	💡 Idea	—	—	—
Synchronous vs Asynchronous AI	💡 Idea	—	—	—
Queues & Workers	💡 Idea	—	—	—
Caching Strategies	💡 Idea	—	—	—
Database Architecture	💡 Idea	—	—	—
Vector Database Architecture	💡 Idea	—	—	—
Model Routing	💡 Idea	—	—	—
Multi-Model Systems	💡 Idea	—	—	—
Reliability	💡 Idea	—	—	—
Fault Tolerance	💡 Idea	—	—	—
Cost vs Performance	💡 Idea	—	—	—
Design an AI Platform From Scratch	💡 Idea	—	—	—

Module 12 — Final Capstone: Build a Production AI Platform

Lesson	Status	Script	Code	Published
Phase 1 — Requirements	💡 Idea	—	—	—
Phase 2 — Architecture	💡 Idea	—	—	—
Phase 3 — Repository	💡 Idea	—	—	—
Phase 4 — Backend	💡 Idea	—	—	—
Phase 5 — AI Layer	💡 Idea	—	—	—
Phase 6 — Agent	💡 Idea	—	—	—
Phase 7 — RAG	💡 Idea	—	—	—
Phase 8 — MCP	💡 Idea	—	—	—
Phase 9 — Multi-Agent	💡 Idea	—	—	—
Phase 10 — Production	💡 Idea	—	—	—
Phase 11 — Infrastructure	💡 Idea	—	—	—

Lesson	Status	Script	Code	Published
Phase 12 — Deployment	💡 Idea	—	—	—
Phase 13 — Final Architecture Review	💡 Idea	—	—	—

10 · Weekly Operating System

Sunday — Plan

- Choose 2 long-form videos
- Choose 4 Shorts
- Choose project tasks
- Research upcoming topics

Monday–Saturday — Execute

Sunday — Review

- What was completed?
- What wasn't?
- Why?
- What should move to next week?
- What performed well?
- What should be repeated?

11 · Monthly Goals

Content

- 8 long-form videos
- 16–24 Shorts
- 1 major project milestone

Course

- Complete approximately 1 module or major section

Engineering

- Build corresponding GitHub projects
- Improve examples
- Refactor code

12 · Course Quality Checklist

Before publishing any lesson:

- Concept is clearly explained
- Real-world example included
- Code works
- Architecture explained
- Common mistakes discussed
- Production considerations included
- GitHub code available
- Resources linked
- Video title optimized
- Thumbnail created
- Short ideas extracted
- Lesson linked to capstone where relevant

13 · DynamoAI Long-Term Roadmap

Phase 1 — Foundation (Months 1–3)

- Launch YouTube
- Build initial audience
- Publish AI/backend tutorials
- Start Python prerequisite content
- Start flagship course

Phase 2 — Authority (Months 4–6)

- AI Agents
- RAG
- MCP
- System Design
- Production AI
- More end-to-end projects

Phase 3 — Flagship (Months 7–9)

- Complete AI Engineering course
- Complete capstone
- Publish GitHub repository
- Create full YouTube playlist

Phase 4 — Expansion (Months 10–12)

- Backend Engineering course
- AI System Design course
- Advanced Agent Engineering
- Career content
- Community

The Most Important Rule

Don't try to build all of this at once. Your immediate priority:

- Step 1 — Finalize DynamoAI branding
- Step 2 — Set up the Notion workspace (or this doc as fallback)
- Step 3 — Put the 12-module curriculum into the Curriculum tracker
- Step 4 — Create the GitHub course repository
- Step 5 — Prepare the first 10 YouTube videos
- Step 6 — Start recording
- Step 7 — Publish consistently
- Step 8 — Build the flagship course in public

MODULE 1 CONTENT

WEEK 1

#	Lesson	Video	Article
1	Python prerequisites	Python Full Course for Beginners – Mosh	Official Python Tutorial
2	Python project structure	src layout explained – anthony explains	Structuring Your Project – Hitchhiker's Guide
3	Functions	Python Tutorial for Beginners 8: Functions – Corey Schafer	Defining Your Own Python Function – Real Python
4	Type hints	Type Hints: Basic to Generics – Corey Schafer	Python Type Checking – Real Python
5	Classes & OOP	OOP Tutorial 1: Classes and Instances – Corey Schafer	Object-Oriented Programming in Python – Real Python
6	Dataclasses	Neat Tricks for Customizing Dataclasses – ArjanCodes	Data Classes in Python – Real Python
7	Pydantic	Parsing Data with Pydantic – Michael Kennedy (about 5 min)	Pydantic guide – Real Python
8	Error handling	Try/Except Blocks for Error Handling – Corey Schafer	Python Exceptions – Real Python
9	JSON	Working with JSON Data – Corey Schafer	Working With JSON Data – Real Python
10	HTTP requests	Python Requests Tutorial – Corey Schafer	Python's Requests Library – Real Python
11	Environment variables	Manage Environment Variables (.env + dotenv)	python-dotenv docs
12	Virtual environments	venv (Mac & Linux) – Corey Schafer	Virtual Environments Primer – Real Python
13	Dependencies	UV: All-in-One Package Manager – Corey Schafer	Managing Projects With uv – Real Python

14	Async Python	AsyncIO Complete Guide with Animations – Corey Schafer	Conceptual Overview of asyncio – Python docs
15	async / await	AsyncIO & Async/Await – Tech With Tim	Async IO in Python – Real Python
16	Testing with pytest	Beginner's Guide to Pytest – Teclado	Effective Testing with pytest – Real Python
17	Testing with pytest	Beginner's Guide to Pytest – Teclado	Effective Testing with pytest – Real Python

SLIDE: LINK

MODUEL 2

#	Lesson	🎥 Video	📖 Article
1	How the Internet Works	How does the INTERNET work? – Lesics	How does the Internet work? – MDN
2	HTTP	HTTP Crash Course & Exploration – Traversy Media	An overview of HTTP – MDN
3	REST APIs	What is a REST API? – IBM Technology	What Is a REST API? – IBM
4	API Design	REST API Design Best Practices and Guidelines	Web API Design Best Practices – Microsoft
5	FastAPI	FastAPI Part 1: Getting Started – Corey Schafer	FastAPI Official Tutorial
6	Routes	FastAPI Part 3: Path Parameters – Corey Schafer	Path Parameters – FastAPI docs
7	Request Validation	FastAPI Part 4: Pydantic Schemas – Corey Schafer	Request Body – FastAPI docs
8	Response Models	FastAPI Part 4: Pydantic Schemas – Corey Schafer	Response Model – FastAPI docs
9	Dependency Injection	Dependency Injection Explained Like You're 5 (FastAPI)	Dependencies – FastAPI docs
10	Authentication	FastAPI Part 10: Registration & Login with JWT – Corey Schafer	Security – FastAPI docs
11	Authorization	FastAPI Part 11: Protecting Routes – Corey Schafer	Authorization Cheat Sheet – OWASP
12	PostgreSQL	Learn PostgreSQL – freeCodeCamp	PostgreSQL Official Tutorial
13	SQL	SQL Full Course for Beginners – freeCodeCamp	SQLBolt (interactive lessons)
14	Database Design	Database Design Course – freeCodeCamp	Learn Relational Database Design – freeCodeCamp
15	SQLAlchemy	FastAPI Part 5: SQLAlchemy Models – Corey Schafer	SQLAlchemy Unified Tutorial
16	Database Migrations	FastAPI Part 15: PostgreSQL & Alembic – Corey Schafer	Alembic Tutorial

17	Redis	Redis in 100 Seconds – Fireship	How to Use Redis With Python – Real Python
18	Caching	Cache Systems Every Developer Should Know – ByteByteGo	Caching Overview – AWS
19	Background Jobs	Background Tasks & Celery + Redis – FastAPI Beyond CRUD	Background Tasks – FastAPI docs
20	Queues	Message Queue Fundamentals in 20 Minutes	Async Tasks with FastAPI and Celery – TestDriven.io
21	Async Backend Architecture	FastAPI Part 7: Sync vs Async – Corey Schafer	Concurrency and async/await – FastAPI docs

MODULE 1 CONTENT

WEEK 1

#	Lesson	Video	Article
1	Python prerequisites	Python Full Course for Beginners – Mosh	Official Python Tutorial
2	Python project structure	src layout explained – anthony explains	Structuring Your Project – Hitchhiker's Guide
3	Functions	Python Tutorial for Beginners 8: Functions – Corey Schafer	Defining Your Own Python Function – Real Python
4	Type hints	Type Hints: Basic to Generics – Corey Schafer	Python Type Checking – Real Python
5	Classes & OOP	OOP Tutorial 1: Classes and Instances – Corey Schafer	Object-Oriented Programming in Python – Real Python
6	Dataclasses	Neat Tricks for Customizing Dataclasses – ArjanCodes	Data Classes in Python – Real Python
7	Pydantic	Parsing Data with Pydantic – Michael Kennedy (about 5 min)	Pydantic guide – Real Python
8	Error handling	Try/Except Blocks for Error Handling – Corey Schafer	Python Exceptions – Real Python
9	JSON	Working with JSON Data – Corey Schafer	Working With JSON Data – Real Python

SLIDE: LINK

Week 1 Exercise: AI Model Catalog

Build a small Python program that loads a catalog of AI models from a JSON file, rejects the bad records, answers questions about the good ones and saves a summary report. It uses all nine Week 1 lessons and takes about 2–3 hours.

What you'll practise

Each Week 1 lesson has a job in this project.

#	Lesson	Where you use it
1	Python prerequisites	Loops, lists, dicts and f-strings throughout
2	Project structure	A `src/` package with separate modules
3	Functions	`load_models()` and `main()`
4	Type hints	Every function and method is fully typed
5	Classes & OOP	The `ModelCatalog` class and its methods
6	Dataclasses	The `CatalogSummary` report object
7	Pydantic	The `ModelInfo` model that validates each record
8	Error handling	Skipping bad records; handling a missing or broken file
9	JSON	Reading `models.json` and writing `report.json`

Set up the project

Create this layout, then install Pydantic with `pip install pydantic`.

```
model_catalog/
├── data/
│   └── models.json
└── src/
    └── model_catalog/
        ├── __init__.py
        ├── schemas.py   # ModelInfo (Pydantic) + CatalogSummary (dataclass)
        ├── loader.py    # load_models()
        ├── catalog.py   # ModelCatalog class
        └── main.py      # main(): runs everything
```

Copy this into `data/models.json` exactly as it is. The model names are made up, and **three records are broken on purpose**. Your program must catch them.

```
[
  {"name": "Atlas-S", "provider": "Northwind AI", "context_window": 128000, "price_per_million_tokens": 0.5,
"open_weights": false},
  {"name": "Atlas-L", "provider": "Northwind AI", "context_window": 200000, "price_per_million_tokens": 3.0,
"open_weights": false},
  {"name": "Kestrel-7B", "provider": "OpenField", "context_window": 32000, "price_per_million_tokens": 0.1,
"open_weights": true},
  {"name": "Kestrel-70B", "provider": "OpenField", "context_window": 128000, "price_per_million_tokens": 0.9,
"open_weights": true},
  {"name": "Lumen", "provider": "Brightline", "context_window": "1M", "price_per_million_tokens": 2.0, "open_weights":
false},
  {"name": "Orca-Mini", "provider": "DeepHarbor", "context_window": 16000, "price_per_million_tokens": -1,
"open_weights": true},
  {"provider": "Brightline", "context_window": 64000, "price_per_million_tokens": 1.2, "open_weights": false},
  {"name": "Sable-Pro", "provider": "Brightline", "context_window": 1000000, "price_per_million_tokens": 5.0,
"open_weights": false}
]
```

Run the program from the `model_catalog/` folder:

- macOS / Linux: `PYTHONPATH=src python -m model_catalog.main`
- Windows (PowerShell): `$env:PYTHONPATH="src"; python -m model_catalog.main`

Your tasks

Work through these in order. Each step builds on the one before.

1. **Validate one record with Pydantic** (`schemas.py`). Create a `ModelInfo` class that inherits from `BaseModel`, with five fields:
    - `name` and `provider`: text that can't be empty
    - `context_window`: a whole number above 0
    - `price_per_million_tokens`: a decimal number, 0 or more
    - `open_weights`: true or false
2.
    Hint: use `Field(min_length=1)`, `Field(gt=0)` and `Field(ge=0)`.

3. **Describe the report with a dataclass** (`schemas.py`). Create a `CatalogSummary` dataclass with these fields: `total_models`, `skipped_records`, `open_weight_count`, `cheapest_model`, `largest_context_model` and `average_price`. Add a type hint to each one. The two model names can be `None` when the catalog is empty.
4. **Load the file safely** (`loader.py`). Write `load_models(path: Path) -> tuple[list[ModelInfo], int]`. It reads the JSON file and tries to turn each record into a `ModelInfo`. When a record fails, catch the `ValidationError`, print which record and field was wrong, then carry on. Return the valid models and the number you skipped.
5. **Build the catalog class** (`catalog.py`). Create `ModelCatalog`, which receives the list of models in `__init__`. Give it these methods, all with type hints:
    - `open_weight_models()`: returns the models whose weights are open
    - `cheapest()`: returns the model with the lowest price, or `None` if there are no models
    - `largest_context()`: returns the model with the biggest context window, or `None` if there are no models
    - `with_min_context(min_tokens)`: returns the models with at least that much context
    - `summary()`: returns a filled-in `CatalogSummary`, with `average_price` rounded to 2 decimals
6. **Handle file problems** (`main.py`). In `main()`, call `load_models()` inside a `try` block. If the file is missing (`FileNotFoundError`) or isn't valid JSON (`json.JSONDecodeError`), print a clear message and stop. The program must not crash with a traceback.
7. **Print and save the results** (`main.py`). Print the counts, the cheapest model, the model with the largest context window, and every model with at least 128,000 tokens of context. Then save the summary to `report.json` using `dataclasses.asdict()` and `json.dump(..., indent=2)`.

Expected output

When it works, you should get 5 valid models and 3 skipped. Your messages can be worded differently, but the numbers and names must match.

```
Skipping record 4: context_window - Input should be a valid integer, unable to parse string as an integer
Skipping record 5: price_per_million_tokens - Input should be greater than or equal to 0
Skipping record 6: name - Field required

Loaded 5 models, skipped 3.
Cheapest: Kestrel-7B
Largest context window: Sable-Pro
Models with at least 128,000 tokens of context:
  - Atlas-S (Northwind AI)
  - Atlas-L (Northwind AI)
  - Kestrel-70B (OpenField)
  - Sable-Pro (Brightline)

Report saved to report.json
```

`report.json` should contain:

```
{
  "total_models": 5,
  "skipped_records": 3,
  "open_weight_count": 2,
  "cheapest_model": "Kestrel-7B",
  "largest_context_model": "Sable-Pro",
  "average_price": 1.9
}
```

To test your error handling, rename `models.json` and run the program again. Then put the file back, delete its last `]`, and run it once more. Both times you should see a one-line error message, not a traceback.

Before you submit

- The folder layout matches the setup section, with your code inside `src/model_catalog/`
- Every function and method has type hints for its inputs and its return value
- Bad records are skipped with a message, and the program keeps going
- A missing file or broken JSON prints a clear error instead of a traceback
- Your terminal output and `report.json` match the expected output
- No code is copied and pasted twice: repeated logic lives in a function or method